

Cloud-Based Learning Platform - Complete Documentation

Table of Contents

1. Executive Summary
 2. System Architecture
 3. Service Catalog
 4. API Documentation
 5. Event-Driven Architecture
 6. Database Schemas
 7. SOLID Principles Application
 8. Testing & Verification
 9. Containerization & Deployment
 10. Complete Workflows
 11. Troubleshooting & Known Issues
 12. Architecture Compliance Checklist
 13. Future Enhancements
 14. Conclusion
-

1. Executive Summary

1.1 Platform Overview

The Cloud-Based Learning Platform is a **production-grade, cloud-native microservices architecture** designed for educational content processing, quiz generation, conversational AI, and multimedia services. The platform follows **event-driven architecture** principles, ensuring scalability, resilience, and maintainability.

1.2 Key Features

Feature	Status	Description
Document Processing	✓ Production Ready	PDF, DOCX, TXT upload and processing with AI-powered note generation
Quiz Generation	✓ Production Ready	AI-generated quizzes from documents using HuggingFace models
Conversational AI	✓ Production Ready	RAG-powered chat service with Gemini 1.5 Flash
Text-to-Speech	✓ Production Ready	Google TTS integration for audio generation
Speech-to-Text	✓ Production Ready	Gemini 1.5 Flash multimodal transcription
Real-time Notifications	✓ Production Ready	Event-driven notification system
User Management	✓ Production Ready	JWT-based authentication and authorization

1.3 Technology Stack

Layer	Technology
API Gateway	Nginx
Backend Framework	FastAPI (Python), Express.js (Node.js)
Message Broker	Apache Kafka
Databases	PostgreSQL (with pgvector), MongoDB
Object Storage	AWS S3 (LocalStack for local dev)
AI Services	Google Gemini 1.5 Flash, HuggingFace Inference API
Containerization	Docker, Docker Compose
Orchestration	Kubernetes (planned)

1.4 Architecture Principles

- **Microservices:** Each service is independently deployable and scalable
- **Event-Driven:** Asynchronous communication via Kafka
- **Storage Isolation:** Each service has its own database and S3 bucket
- **SOLID Principles:** Applied throughout the codebase
- **12-Factor App:** Environment-based configuration, stateless services

2. System Architecture

2.1 High-Level Architecture

Architecture Layers:

1. **Client Layer:** Browser, Mobile App, API Client
2. **Edge/Ingress Layer:** Nginx API Gateway (Port 80)
3. **Application Layer:** Docker Compose/Kubernetes Cluster
 - Aggregator Service (BFF - Port 8080)
 - Microservices Cluster (11 services)
 - Kafka Cluster (3 brokers + Zookeeper)
4. **Storage Layer:**
 - PostgreSQL Databases (5 instances)
 - MongoDB Databases (2 instances)
 - LocalStack S3 (document-reader-storage-dev)

Service Communication:

- **Synchronous:** HTTP REST APIs (through Aggregator)
- **Asynchronous:** Kafka events for long-running operations

2.2 Service Communication Patterns

2.2.1 Synchronous Communication (HTTP)

From	To	Endpoint	Purpose	Response Time
Frontend	API Gateway	POST /api/*	All API requests	< 100ms
API Gateway	Aggregator	POST /api/*	Route to aggregator	< 50ms
Aggregator	User Service	GET /api/auth/verify	JWT verification	< 200ms
Aggregator	Document Service	POST /api/documents/upload	Upload document	< 2s
Aggregator	Chat Service	POST /api/chat/message	Get chat response	< 5s
Aggregator	Quiz Service	GET /api/quizzes/{id}	Get quiz	< 500ms

2.2.2 Asynchronous Communication (Kafka)

All long-running operations use Kafka for async processing:

- **Document Processing:** Upload → Process → Generate Notes
- **Quiz Generation:** Request → Generate → Complete

- **Audio Generation:** Request → Synthesize → Complete
- **Audio Transcription:** Request → Transcribe → Complete

2.3 Storage Architecture

Storage Isolation Principle: Each service has its own database and S3 bucket prefix.

Service	Database	S3 Bucket/Prefix
User Service	<code>user_management</code> (PostgreSQL)	N/A
Document Service	<code>document_reader_db</code> (PostgreSQL)	<code>document-reader-storage-dev/documents/</code>
Quiz Service	<code>quiz_db</code> (PostgreSQL)	<code>document-reader-storage-dev/quizzes/</code>
Chat Service	<code>chat_db</code> (PostgreSQL + pgvector)	<code>document-reader-storage-dev/embeddings/</code>
TTS Service	<code>tts_service</code> (MongoDB)	<code>document-reader-storage-dev/audio/</code>
STT Service	<code>stt_service</code> (MongoDB)	<code>document-reader-storage-dev/audio/</code>
Notification Service	<code>notification_db</code> (PostgreSQL)	N/A

3. Service Catalog

3.1 User Service

Port: 8000

Language: Python 3.11

Framework: FastAPI

Database: PostgreSQL (`user_management`)

3.1.1 Responsibilities

- User registration and authentication
- JWT token generation and validation
- User profile management
- Password hashing (bcrypt)

3.1.2 API Endpoints

Method	Endpoint	Description	Auth Required
<code>POST</code>	<code>/api/auth/register</code>	Register new user	No
<code>POST</code>	<code>/api/auth/login</code>	Login and get JWT token	No
<code>GET</code>	<code>/api/auth/verify</code>	Verify JWT token	Yes

Method	Endpoint	Description	Auth Required
GET	/health	Health check	No

3.1.3 Request/Response Examples

Register User:

bash

```
POST http://localhost:8000/api/auth/register
Content-Type: application/json

{
  "username": "john_doe",
  "email": "john@example.com",
  "password": "SecurePass123!"
}
```

Response:

json

```
{
  "user_id": "f243ed10-5b07-4e5c-b9ee-bd668ea257ac",
  "username": "john_doe",
  "email": "john@example.com",
  "message": "User registered successfully"
}
```

Login:

bash

```
POST http://localhost:8000/api/auth/login
Content-Type: application/json

{
  "email": "john@example.com",
  "password": "SecurePass123!"
}
```

Response:

json

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "bearer",
  "user_id": "f243ed10-5b07-4e5c-b9ee-bd668ea257ac",
}
```

```
"username": "john_doe"
}
```

3.1.4 Database Schema

sql

```
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  username VARCHAR(50) UNIQUE NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_users_username ON users(username);
```

3.1.5 Testing

PowerShell Test:

powershell

```
# Register
$registerBody = @{
  username = "testuser"
  email = "test@example.com"
  password = "Test123!"
} | ConvertTo-Json

$registerResponse = Invoke-RestMethod -Uri "http://localhost:8000/api/auth/register" `
  -Method POST -Body $registerBody -ContentType "application/json"

# Login
$loginBody = @{
  email = "test@example.com"
  password = "Test123!"
} | ConvertTo-Json

$loginResponse = Invoke-RestMethod -Uri "http://localhost:8000/api/auth/login" `
  -Method POST -Body $loginBody -ContentType "application/json"

$global:token = $loginResponse.access_token
$global:userId = $loginResponse.user_id

Write-Host "Token: $global:token"
Write-Host "User ID: $global:userId"
```

Expected Output:

text

```
Token: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoizjI0M2VkMTAtNWlwny00ZTVjLWJ...
User ID: f243ed10-5b07-4e5c-b9ee-bd668ea257ac
```

3.2 Aggregator Service (BFF)

Port: 8080

Language: Python 3.11

Framework: FastAPI

Database: None (stateless)

3.2.1 Responsibilities

- **Backend for Frontend (BFF):** Single entry point for all client requests
- **JWT Authentication:** Verifies tokens using shared security module
- **Request Orchestration:** Routes requests to appropriate microservices
- **Kafka Command Publishing:** Publishes command events for async operations
- **Response Aggregation:** Combines responses from multiple services (future)

3.2.2 API Endpoints

Method	Endpoint	Description	Auth Required
GET	/health	Health check	No
GET	/api/test-auth	Test JWT authentication	Yes
POST	/api/documents/upload	Upload document (proxies to Document Service)	Yes
GET	/api/documents	List documents (proxies to Document Service)	Yes
GET	/api/documents/{id}	Get document (proxies to Document Service)	Yes
GET	/api/documents/{id}/notes	Get notes (proxies to Document Service)	Yes
POST	/api/documents/{id}/regenerate-notes	Regenerate notes (proxies to Document Service)	Yes
DELETE	/api/documents/{id}	Delete document (proxies to Document Service)	Yes

Method	Endpoint	Description	Auth Required
POST	/api/quiz/generate	Request quiz generation (Kafka command)	Yes
GET	/api/quizzes/{id}	Get quiz (proxies to Quiz Service)	Yes
GET	/api/quizzes	List quizzes (proxies to Quiz Service)	Yes
POST	/api/chat/message	Send chat message (proxies to Chat Service)	Yes
POST	/api/tts/synthesize	Request TTS generation (Kafka command)	Yes
POST	/api/stt/transcribe	Request STT transcription (Kafka command)	Yes

3.2.3 Communication Patterns

HTTP Proxying (Document Service):

- **Why?** Maintains strict storage isolation
- **How:** Aggregator proxies file uploads to Document Service via HTTP
- **Benefit:** Only Document Service has write access to its S3 bucket

Kafka Commands (Quiz, TTS, STT):

- **Why?** Async processing for long-running operations
- **How:** Aggregator publishes command events to Kafka
- **Flow:** Command → Service processes → Completion event → Notification Service → User

3.2.4 Kafka Producer Initialization

The Aggregator includes a Kafka Producer initialized at startup using FastAPI's lifespan:

python

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    global kafka_producer

    # Startup: Initialize Kafka Producer with retry logic
    max_retries = 5
    retry_delay = 2 # seconds

    for attempt in range(max_retries):
        try:
            kafka_producer = KafkaProducer(
                bootstrap_servers=KAFKA_BOOTSTRAP_SERVERS,
```



```

        value_serializer=lambda v: json.dumps(v).encode('utf-8'),
        key_serializer=lambda k: k.encode('utf-8') if k else None,
        acks='all',
        retries=3,
        request_timeout_ms=10000,
    )
    logger.info("✅ Kafka Producer initialized successfully")
    break
except Exception as e:
    if attempt < max_retries - 1:
        await asyncio.sleep(retry_delay)
    else:
        logger.error("❌ Failed to initialize Kafka Producer")
        kafka_producer = None

yield # Application runs here

# Shutdown: Close Producer
if kafka_producer:
    kafka_producer.close(timeout=5)

```

3.2.5 Testing

Test Authentication:

powershell

```

Invoke-RestMethod -Uri "http://localhost:8080/api/test-auth" `
    -Method GET -Headers @{Authorization="Bearer $global:token"}

```

Expected Output:

json

```

{
  "authenticated": true,
  "user_id": "f243ed10-5b07-4e5c-b9ee-bd668ea257ac",
  "username": "testuser"
}

```

3.3 Document Service

Port: 8002 (external), 8000 (internal)

Language: Python 3.11

Framework: FastAPI

Database: PostgreSQL (`document_reader_db`)

Storage: S3 (`document-reader-storage-dev`)

3.3.1 Responsibilities

- Document upload and storage (PDF, DOCX, TXT)
- Text extraction from documents
- AI-powered note generation using Gemini 1.5 Flash
- Document metadata management
- S3 file management

3.3.2 API Endpoints

Method	Endpoint	Description	Auth Required
POST	/api/documents/upload	Upload document	Yes
GET	/api/documents	List user's documents	Yes
GET	/api/documents/{id}	Get document metadata	Yes
GET	/api/documents/{id}/notes	Get generated notes	Yes
POST	/api/documents/{id}/regenerate-notes	Regenerate notes	Yes
DELETE	/api/documents/{id}	Delete document	Yes
GET	/health	Health check	No

3.3.3 Document Processing Flow

1. User uploads document via Aggregator
2. Document Service receives file
3. Upload to S3 (raw file)
4. Save metadata to PostgreSQL (status: UPLOADED)
5. Publish Kafka event: document.uploaded
6. Document Worker consumes event
7. Download from S3
8. Extract text (PDF/DOCX/TXT)
9. Upload extracted text to S3
10. Generate notes using Gemini 1.5 Flash
11. Upload notes to S3
12. Update database (status: PROCESSED)
13. Publish Kafka events: document.processed, notes.generated

3.3.4 Database Schema

sql

```
CREATE TABLE documents (  
  id UUID PRIMARY KEY,  
  user_id VARCHAR(64) NOT NULL,  
  filename VARCHAR(255) NOT NULL,  
  original_filename VARCHAR(255),  
  s3_raw_url TEXT,  
  s3_text_url TEXT,  
  s3_notes_url TEXT,  
  status VARCHAR(50) DEFAULT 'UPLOADED',  
  file_size INTEGER,  
  file_type VARCHAR(100),  
  pages INTEGER,  
  text_length INTEGER,  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),  
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()  
);  
  
CREATE INDEX idx_documents_user_id ON documents(user_id);  
CREATE INDEX idx_documents_status ON documents(status);
```

3.3.5 AI Integration

Model: Gemini 1.5 Flash

API Key: `GOOGLE_API_KEY`

Endpoint: `https://generativelanguage.googleapis.com/v1beta/models/gemini-1.5-flash:generateContent`

Note Generation Prompt:

text

You are an expert educational AI. Generate comprehensive study notes from the following document text.

Document Text:
{extracted_text}

Instructions:
1. Create structured notes with clear sections
2. Highlight key concepts and definitions
3. Include important examples
4. Keep notes concise but comprehensive
5. Use markdown formatting for structure

Generate the notes now:

3.3.6 Testing

Upload Document:

powershell

```
$filePath = "C:\path\to\document.pdf"
$fileBytes = [System.IO.File]::ReadAllBytes($filePath)
$boundary = [System.Guid]::NewGuid().ToString()
$bodyLines = @(
    "--$boundary",
    "Content-Disposition: form-data; name=`file`; filename=`document.pdf`",
    "Content-Type: application/pdf",
    "",
    [System.Text.Encoding]::GetEncoding("iso-8859-1").GetString($fileBytes),
    "--$boundary--"
) -join "`r`n"

$headers = @{
    Authorization = "Bearer $global:token"
    "Content-Type" = "multipart/form-data; boundary=$boundary"
}

$response = Invoke-RestMethod -Uri "http://localhost/api/documents/upload" `
    -Method POST -Body $bodyLines -Headers $headers

$global:documentId = $response.document_id
Write-Host "Document ID: $global:documentId"
```

Expected Output:

json

```
{
  "document_id": "3af31b69-5a41-49de-b656-1b9b296b7744",
  "filename": "document.pdf",
  "status": "UPLOADED",
  "message": "Document uploaded successfully. Processing will begin shortly."
}
```

Check Processing Status:

powershell

```
# Wait 30 seconds for processing
Start-Sleep -Seconds 30

# Get document details
$doc = Invoke-RestMethod -Uri "http://localhost/api/documents/$global:documentId" `
    -Method GET -Headers @{Authorization="Bearer $global:token"}

Write-Host "Status: $($doc.status)"
Write-Host "Pages: $($doc.pages)"
Write-Host "Text Length: $($doc.text_length)"
```

Get Notes:

powershell

```
$notes = Invoke-RestMethod -Uri "http://localhost/api/documents/$global:documentId/notes" `
    -Method GET -Headers @{Authorization="Bearer $global:token"}

Write-Host "Notes URL: $($notes.notes_url)"
```

3.4 Document Worker

Type: Background Worker

Language: Python 3.11

Database: PostgreSQL (`document_reader_db`)

Storage: S3

3.4.1 Responsibilities

- Consume `document.uploaded` Kafka events
- Extract text from documents (PDF, DOCX, TXT)
- Generate AI-powered notes using Gemini 1.5 Flash
- Upload processed files to S3
- Publish completion events to Kafka

3.4.2 Kafka Topics

Consumes:

- `document.uploaded` (Consumer Group: `document-service-group`)

Produces:

- `document.processed`
- `notes.generated`

3.4.3 Processing Logic

python

```
def process_document(document_id: str, s3_url: str):
    # 1. Download from S3
    file_content = s3_service.download_file(s3_url)

    # 2. Extract text based on file type
```

```

if file_type == "application/pdf":
    text = extract_text_from_pdf(file_content)
elif file_type == "application/vnd.openxmlformats-officedocument.wordprocessingml.document":
    text = extract_text_from_docx(file_content)
elif file_type == "text/plain":
    text = file_content.decode('utf-8')

# 3. Upload extracted text to S3
text_s3_url = s3_service.upload_file(text.encode('utf-8'), f"{document_id}/text.txt")

# 4. Generate notes using Gemini
notes = ai_service.generate_notes(text)

# 5. Upload notes to S3
notes_s3_url = s3_service.upload_file(notes.encode('utf-8'), f"{document_id}/notes.md")

# 6. Update database
document.status = "PROCESSED"
document.s3_text_url = text_s3_url
document.s3_notes_url = notes_s3_url

# 7. Publish events
kafka_service.produce_document_processed(document_id, text_s3_url)
kafka_service.produce_notes_generated(document_id, notes_s3_url)

```

3.4.4 Testing

Monitor Worker Logs:

powershell

```
docker-compose logs -f document-worker
```

Expected Output:

text

```

document-worker | INFO: Consuming from topic: document.uploaded
document-worker | INFO: Processing document: 3af31b69-5a41-49de-b656-1b9b296b7744
document-worker | INFO: Extracted 5000 characters from PDF
document-worker | INFO: Generating notes with Gemini...
document-worker | INFO: Notes generated successfully
document-worker | INFO: Published document.processed event
document-worker | INFO: Published notes.generated event

```

3.5 Quiz Service

Port: 8004

Language: Python 3.11

Framework: FastAPI

Database: PostgreSQL (`quiz_db`)

Storage: S3 (`document-reader-storage-dev/quizzes/`)

3.5.1 Responsibilities

- Quiz generation from document text using HuggingFace models
- Quiz storage and retrieval
- Quiz metadata management

3.5.2 API Endpoints

Method	Endpoint	Description	Auth Required
GET	<code>/api/quizzes/{id}</code>	Get quiz by ID	Yes
GET	<code>/api/quizzes</code>	List quizzes for user	Yes
GET	<code>/health</code>	Health check	No

3.5.3 Quiz Generation Flow

1. User requests quiz via Aggregator
2. Aggregator publishes Kafka event: `quiz.requested`
3. Quiz Worker consumes event
4. Download document text from S3
5. Call HuggingFace API (Qwen/Qwen2.5-7B-Instruct)
6. Parse JSON response
7. Save quiz to PostgreSQL
8. Upload quiz JSON to S3
9. Publish Kafka event: `quiz.generated`
10. Notification Service notifies user

3.5.4 AI Integration

Model: `Qwen/Qwen2.5-7B-Instruct`

API: HuggingFace Inference API

Endpoint: `https://api-inference.huggingface.co/models/Qwen/Qwen2.5-7B-Instruct`

Method: `chat_completion`

Quiz Generation Prompt:

text

You are an expert educational AI. Generate a quiz based strictly on the provided text.

Text Content:
{document_text}

Instructions:

1. Generate 5 multiple-choice questions.
2. Difficulty Level: {difficulty}.
3. Each question must have exactly 4 options.
4. Provide the correct answer index (0-3).
5. Provide a short explanation.

Output MUST be a single valid JSON object with this exact schema:

```
{
  "title": "Quiz Title",
  "difficulty": "{difficulty}",
  "questions": [
    {
      "question_text": "Question text?",
      "options": ["Option A", "Option B", "Option C", "Option D"],
      "correct_answer_index": 0,
      "explanation": "Reasoning here."
    }
  ]
}
```

Do not add any text outside the JSON. Return only the JSON object.

3.5.5 Database Schema

sql

```
CREATE TABLE quizzes (
  id UUID PRIMARY KEY,
  user_id VARCHAR(64) NOT NULL,
  document_id UUID NOT NULL,
  title VARCHAR(255) NOT NULL,
  difficulty VARCHAR(50),
  question_count INTEGER,
  s3_url TEXT,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE TABLE questions (
  id UUID PRIMARY KEY,
  quiz_id UUID REFERENCES quizzes(id) ON DELETE CASCADE,
  question_text TEXT NOT NULL,
  options JSONB NOT NULL,
  correct_answer_index INTEGER NOT NULL,
  explanation TEXT,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```



```
);

CREATE INDEX idx_quizzes_user_id ON quizzes(user_id);
CREATE INDEX idx_quizzes_document_id ON quizzes(document_id);
CREATE INDEX idx_questions_quiz_id ON questions(quiz_id);
```

3.5.6 Testing

Request Quiz Generation:

powershell

```
$quizRequest = Invoke-RestMethod -Uri "http://localhost/api/quiz/generate?document_id=$global:documentId&difficulty=medium&question_count=10" `
    -Method POST -Headers @{Authorization="Bearer $global:token"}

Write-Host "Request ID: $($quizRequest.request_id)"
Write-Host "Status: $($quizRequest.status)"
```

Expected Output:

json

```
{
  "request_id": "req-12345",
  "document_id": "3af31b69-5a41-49de-b656-1b9b296b7744",
  "status": "accepted",
  "message": "Quiz generation request accepted. Processing will begin shortly."
}
```

Wait and Get Quiz:

powershell

```
# Wait 60 seconds for generation
Start-Sleep -Seconds 60

# Get quiz
$quiz = Invoke-RestMethod -Uri "http://localhost/api/quizzes?user_id=$global:userId&document_id=$global:documentId" `
    -Method GET -Headers @{Authorization="Bearer $global:token"}

$quiz[0] | ConvertTo-Json -Depth 10
```

Expected Output:

json

```
{
  "id": "quiz-12345",
  "title": "Quiz on Document Content",
  "document_id": "3af31b69-5a41-49de-b656-1b9b296b7744",
  "questions": [
    {
      "id": "q1",
      "question_text": "What is the main topic of this document?",
      "options": [
        "Option A",
        "Option B",
        "Option C",
        "Option D"
      ]
    }
  ]
},
"created_at": "2025-01-XXT12:00:00Z"
}
```

3.6 Quiz Worker

Type: Background Worker

Language: Python 3.11

Database: PostgreSQL (`quiz_db`)

AI Service: HuggingFace Inference API

3.6.1 Responsibilities

- Consume `quiz.requested` Kafka events
- Consume `document.processed` events (for auto-quiz generation)
- Generate quizzes using HuggingFace models
- Save quizzes to database and S3
- Publish completion events

3.6.2 Kafka Topics

Consumes:

- `quiz.requested` (Consumer Group: `quiz-service-group`)
- `document.processed` (Consumer Group: `quiz-service-group`)

Produces:

- `quiz.generated`

3.6.3 AI Service Implementation

File: `platform/quiz-service/src/services/ai_service.py`

python

```
from huggingface_hub import InferenceClient

class AIService:
    def __init__(self):
        self.api_token = os.getenv("HUGGINGFACE_API_KEY")
        self.model_id = os.getenv("HUGGINGFACE_MODEL", "Qwen/Qwen2.5-7B-Instruct")
        self.client = InferenceClient(token=self.api_token)

    def generate_quiz(self, text_content: str, difficulty: str = "Medium") -> QuizGenerated:
        # Call HuggingFace Chat API
        messages = [{"role": "user", "content": prompt}]
        response = self.client.chat_completion(
            messages=messages,
            model=self.model_id,
            max_tokens=2048,
            temperature=0.1,
            seed=42
        )

        # Extract and parse JSON
        response_text = response.choices[0].message.content
        quiz_data = self._extract_json_from_response(response_text)

        return QuizGenerated(**quiz_data)
```

3.6.4 Model History

Model	Status	Reason
<code>mistralai/Mistral-7B-Instruct-v0.3</code>	❌ Deprecated	Access issues, deprecated endpoint
<code>HuggingFaceH4/zephyr-7b-beta</code>	❌ Not Available	Not supported by free tier provider
<code>Qwen/Qwen2.5-7B-Instruct</code>	✅ Active	Reliable, open model, works with chat_completion

3.6.5 Testing

Monitor Worker Logs:

powershell

```
docker-compose logs -f quiz-worker
```

Expected Output:

text

```
quiz-worker | INFO: Consuming from topics: ['quiz.requested', 'document.processed']
quiz-worker | INFO: Received quiz.requested event for document: 3af31b69-5a41-49de-b656-1b9b296b7744
quiz-worker | INFO: Downloading document text from S3...
quiz-worker | INFO: Invoking chat completion for Qwen/Qwen2.5-7B-Instruct...
quiz-worker | INFO: Successfully generated quiz 'Quiz on Document Content' with 5 questions
quiz-worker | INFO: Published quiz.generated event
```

3.7 Chat Service

Port: 8005

Language: Python 3.11

Framework: FastAPI

Database: PostgreSQL (`chat_db`) with pgvector extension

AI Service: Google Gemini 1.5 Flash

3.7.1 Responsibilities

- RAG-powered conversational AI
- Document vectorization and embedding storage
- Vector similarity search
- Conversation history management
- Context-aware responses using Gemini 1.5 Flash

3.7.2 API Endpoints

Method	Endpoint	Description	Auth Required
POST	/api/chat/message	Send chat message	Yes
GET	/health	Health check	No

3.7.3 RAG Architecture

Retrieval Augmented Generation (RAG):

1. Document Ingestion (Async):

- Worker consumes `document.processed` events
- Downloads text from S3

- Splits text into chunks (1000 chars, 200 overlap)
- Generates embeddings using Google `text-embedding-004`
- Stores vectors in PostgreSQL with pgvector

2. Query Processing (Sync):

- User sends message via Aggregator
- Generate embedding for user query
- Vector search: Find 5 most similar chunks (L2 distance)
- Retrieve conversation history (last 10 messages)
- Send context + query + history to Gemini 1.5 Flash
- Return response to user
- Save conversation to database

3.7.4 Database Schema

sql

```
-- Enable pgvector extension
CREATE EXTENSION IF NOT EXISTS vector;

-- Conversations table
CREATE TABLE conversations (
    id VARCHAR(36) PRIMARY KEY,
    user_id VARCHAR(64) NOT NULL,
    title VARCHAR(255),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Messages table
CREATE TABLE messages (
    id VARCHAR(36) PRIMARY KEY,
    conversation_id VARCHAR(36) REFERENCES conversations(id),
    role VARCHAR(20) NOT NULL, -- 'user' or 'ai'
    content TEXT NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Document embeddings table (for RAG)
CREATE TABLE document_embeddings (
    id VARCHAR(36) PRIMARY KEY,
    document_id VARCHAR(36) NOT NULL,
    content TEXT NOT NULL,
    embedding vector(768), -- 768 dimensions for text-embedding-004
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE INDEX idx_conversations_user_id ON conversations(user_id);
CREATE INDEX idx_messages_conversation_id ON messages(conversation_id);
```

```
CREATE INDEX idx_document_embeddings_document_id ON document_embeddings(document_id);
CREATE INDEX idx_document_embeddings_vector ON document_embeddings USING ivfflat (embedding vector_l2_ops);
```

3.7.5 AI Integration

Chat Model: Gemini 1.5 Flash

Embedding Model: `models/text-embedding-004`

API Key: `CHAT_GOOGLE_API_KEY` (separate from Document Service)

RAG Prompt:

text

```
You are a helpful AI tutor for a learning platform.
Use the following pieces of retrieved context to answer the user's question.

Context:
{context_chunks}

Instructions:
1. Answer based strictly on the context provided.
2. If the answer is not in the context, say "I couldn't find that specific information in the document."
3. Keep the answer concise and educational.
4. Use conversation history to understand context and provide coherent follow-up answers.

Conversation History:
{conversation_history}

User Question: {user_query}
```

3.7.6 Testing

Send Chat Message:

powershell

```
$chatBody = @{
    user_id = $global:userId
    document_id = $global:documentId
    message = "What is this document about?"
} | ConvertTo-Json

$chatResponse = Invoke-RestMethod -Uri "http://localhost:8005/api/chat/message" `
    -Method POST -Body $chatBody -ContentType "application/json" `
    -Headers @{Authorization="Bearer $global:token"}

Write-Host "Response: $($chatResponse.response)"
Write-Host "Conversation ID: $($chatResponse.conversation_id)"
Write-Host "Sources Count: $($chatResponse.sources_count)"
```

Expected Output:

json

```
{
  "response": "This document discusses the fundamentals of machine learning, including supervised and u
nsupervised learning algorithms, neural networks, and their applications...",
  "conversation_id": "conv-12345",
  "sources_count": 5
}
```

Follow-up Message:

powershell

```
$followUpBody = @{
    user_id = $global:userId
    document_id = $global:documentId
    message = "Can you explain supervised learning in more detail?"
    conversation_id = $chatResponse.conversation_id
} | ConvertTo-Json

$followUpResponse = Invoke-RestMethod -Uri "http://localhost:8005/api/chat/message" `
    -Method POST -Body $followUpBody -ContentType "application/json" `
    -Headers @{Authorization="Bearer $global:token"}

Write-Host "Response: $($followUpResponse.response)"
```

3.8 Chat Worker

Type: Background Worker

Language: Python 3.11

Database: PostgreSQL (`chat_db`) with pgvector

AI Service: Google `text-embedding-004`

3.8.1 Responsibilities

- Consume `document.processed` Kafka events
- Download document text from S3
- Split text into chunks (RecursiveCharacterTextSplitter)
- Generate embeddings for each chunk
- Store vectors in PostgreSQL with pgvector

3.8.2 Text Splitting Strategy

Library: `langchain_text_splitters.RecursiveCharacterTextSplitter`

Configuration:

- `chunk_size` : 1000 characters
- `chunk_overlap` : 200 characters

Why?

- 1000 chars: Good balance for retrieval (not too small, not too large)
- 200 overlap: Ensures context continuity across chunks

3.8.3 Embedding Generation

Model: `models/text-embedding-004`

Dimensions: 768

API: LangChain `GoogleGenerativeAIEmbeddings`

python

```
from langchain_google_genai import GoogleGenerativeAIEmbeddings

embeddings = GoogleGenerativeAIEmbeddings(
    model="models/text-embedding-004",
    google_api_key=api_key
)

vector = embeddings.embed_query(chunk_text)
```

3.8.4 Testing

Monitor Worker Logs:

powershell

```
docker-compose logs -f chat-worker
```

Expected Output:

text

```
chat-worker | INFO: 🎧 Chat Worker listening for document.processed...
chat-worker | INFO: 🧠 Vectorizing document 3af31b69-5a41-49de-b656-1b9b296b7744...
chat-worker | INFO: Split into 15 chunks. Generating embeddings...
chat-worker | INFO: ✅ Document 3af31b69-5a41-49de-b656-1b9b296b7744 fully ingested (15 vectors)
```


3.9 TTS Service (Text-to-Speech)

Port: 8006

Language: Node.js 18

Framework: Express.js

Database: MongoDB (`tts_service`)

Storage: S3 (`document-reader-storage-dev/audio/`)

3.9.1 Responsibilities

- Text-to-speech synthesis using Google TTS (gTTS)
- Audio file generation (MP3)
- Audio storage in S3
- Kafka event consumption and production

3.9.2 API Endpoints

Method	Endpoint	Description	Auth Required
POST	<code>/api/tts/synthesize</code>	Synthesize text to speech	Yes
GET	<code>/health</code>	Health check	No

3.9.3 TTS Generation Flow

1. User requests TTS via Aggregator
2. Aggregator publishes Kafka event: `audio.generation.requested`
3. TTS Service consumes event
4. Synthesize speech using gTTS
5. Upload audio file to S3
6. Save metadata to MongoDB
7. Publish Kafka event: `audio.generation.completed`
8. Notification Service notifies user

3.9.4 Technology

Library: `gtts` (Google Text-to-Speech)

Output Format: MP3

Language Support: Multiple languages (en, es, fr, etc.)

3.9.5 Database Schema (MongoDB)

javascript

```
{
  _id: ObjectId,
  userId: String,
  text: String,
  audioUrl: String, // S3 URL
  status: String,    // 'pending', 'completed', 'failed'
  languageCode: String,
  createdAt: Date,
  updatedAt: Date
}
```

3.9.6 Testing

Request TTS Generation:

powershell

```
$ttsBody = @{}
    text = "Hello, this is a test of the text to speech system."
    languageCode = "en"
} | ConvertTo-Json

$ttsResponse = Invoke-RestMethod -Uri "http://localhost/api/tts/synthesize" `
    -Method POST -Body $ttsBody -ContentType "application/json" `
    -Headers @{Authorization="Bearer $global:token"}

Write-Host "Request ID: $($ttsResponse.request_id)"
Write-Host "Status: $($ttsResponse.status)"
```

Expected Output:

json

```
{
  "request_id": "req-12345",
  "status": "accepted",
  "message": "TTS generation request accepted. Processing will begin shortly."
}
```

Wait and Check Results:

powershell

```
# Wait 10 seconds
Start-Sleep -Seconds 10
```

```
# Check MongoDB
docker-compose exec -T mongodb mongosh --quiet --eval "use tts_service; db.audios.find().sort({createdAt: -1}).limit(1).pretty()"
```

Expected Output:

json

```
{
  "_id": ObjectId("..."),
  "userId": "f243ed10-5b07-4e5c-b9ee-bd668ea257ac",
  "text": "Hello, this is a test of the text to speech system.",
  "audioUrl": "http://localstack:4566/document-reader-storage-dev/audio/1764965514866-tts/f243ed10-5b07-4e5c-b9ee-bd668ea257ac/1764965514866.mp3",
  "status": "completed",
  "languageCode": "en",
  "createdAt": ISODate("2025-01-XXT12:00:00Z"),
  "updatedAt": ISODate("2025-01-XXT12:00:10Z")
}
```

3.10 STT Service (Speech-to-Text)

Port: 8007

Language: Node.js 18

Framework: Express.js

Database: MongoDB (`stt_service`)

AI Service: Google Gemini 1.5 Flash (Multimodal)

3.10.1 Responsibilities

- Speech-to-text transcription using Gemini 1.5 Flash
- Audio file processing (MP3, WAV, OGG)
- Transcription storage in MongoDB
- Kafka event consumption and production

3.10.2 API Endpoints

Method	Endpoint	Description	Auth Required
POST	/api/stt/transcribe	Transcribe audio to text	Yes
GET	/health	Health check	No

3.10.3 STT Transcription Flow

1. User requests STT via Aggregator
2. Aggregator publishes Kafka event: audio.transcription.requested
3. STT Service consumes event
4. Download audio file from S3
5. Convert audio to Base64
6. Send to Gemini 1.5 Flash (multimodal API)
7. Receive transcription text
8. Save to MongoDB
9. Publish Kafka event: audio.transcription.completed
10. Notification Service notifies user

3.10.4 AI Integration

Model: Gemini 1.5 Flash (Multimodal)

API Key: `GOOGLE_API_KEY` (same as Document Service)

SDK: `@google/generative-ai`

Transcription Implementation:

javascript

```
const { GoogleGenerativeAI } = require("@google/generative-ai");

const genAI = new GoogleGenerativeAI(process.env.GOOGLE_API_KEY);
const model = genAI.getGenerativeModel({ model: "gemini-1.5-flash" });

const result = await model.generateContent([
  {
    inlineData: {
      mimeType: "audio/mp3",
      data: audioBase64
    }
  },
  {
    text: "Please transcribe this audio file accurately. Return ONLY the transcription text."
  }
]);

const transcription = result.response.text();
```

3.10.5 Database Schema (MongoDB)

javascript

```
{
  _id: ObjectId,
  userId: String,
  audioUrl: String, // S3 URL
  transcription: String,
  confidence: Number, // 0.0 - 1.0 (assumed 0.95 for Gemini)
  status: String, // 'pending', 'completed', 'failed'
  languageCode: String,
  createdAt: Date,
  updatedAt: Date
}
```

3.10.6 Testing

Request STT Transcription:

powershell

```
$audioS3Uri = "http://localstack:4566/document-reader-storage-dev/audio/1764965514866-tts/f243ed10-5b07-4e5c-b9ee-bd668ea257ac/1764965514866.mp3"

$sttResponse = Invoke-RestMethod -Uri "http://localhost/api/stt/transcribe?audio_id=test-audio&s3_uri=$([System.Web.HttpUtility]::UrlEncode($audioS3Uri))" `
    -Method POST -Headers @{Authorization="Bearer $global:token"}

Write-Host "Request ID: $($sttResponse.request_id)"
Write-Host "Status: $($sttResponse.status)"
```

Expected Output:

json

```
{
  "request_id": "req-12345",
  "status": "accepted",
  "message": "Transcription request accepted. Processing will begin shortly."
}
```

Wait and Check Results:

powershell

```
# Wait 15 seconds
Start-Sleep -Seconds 15

# Check MongoDB
docker-compose exec -T mongodb mongosh --quiet --eval "use stt_service; db.transcriptions.find().sort({createdAt: -1}).limit(1).pretty()"
```

Expected Output:

json

```
{
  "_id": ObjectId("..."),
  "userId": "f243ed10-5b07-4e5c-b9ee-bd668ea257ac",
  "audioUrl": "http://localhost:4566/document-reader-storage-dev/audio/...",
  "transcription": "Hello, this is a test of the text to speech system.",
  "confidence": 0.95,
  "status": "completed",
  "languageCode": "en-US",
  "createdAt": ISODate("2025-01-XXT12:00:00Z"),
  "updatedAt": ISODate("2025-01-XXT12:00:15Z")
}
```

3.11 Notification Service

Port: 8003

Language: Python 3.11

Framework: FastAPI

Database: PostgreSQL (`notification_db`)

3.11.1 Responsibilities

- Consume all completion events from other services
- Log notifications for audit trail
- Future: WebSocket push notifications
- Future: Email notifications (AWS SES)

3.11.2 Kafka Topics Consumed

Topic	Producer	Purpose
<code>document.processed</code>	Document Worker	Document processing complete
<code>notes.generated</code>	Document Worker	Notes generated from document
<code>quiz.generated</code>	Quiz Worker	Quiz generation complete
<code>audio.generation.completed</code>	TTS Service	Audio generation complete
<code>audio.transcription.completed</code>	STT Service	Transcription complete
<code>chat.message</code>	Chat Service	Chat message sent
<code>user.created</code>	User Service	New user registered

Consumer Group: `notification-service-group`

3.11.3 API Endpoints

Method	Endpoint	Description	Auth Required
GET	/api/notifications	Query notifications	Yes
GET	/api/notifications/{id}	Get notification by ID	Yes
GET	/api/notifications/stats	Get statistics	Yes
GET	/health	Health check	No

3.11.4 Database Schema

sql

```
CREATE TABLE notifications (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id VARCHAR(64),  
  topic VARCHAR(100) NOT NULL,  
  event_type VARCHAR(100) NOT NULL,  
  event_id VARCHAR(100) NOT NULL,  
  payload JSONB NOT NULL,  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()  
);  
  
CREATE INDEX idx_notifications_user_id ON notifications(user_id);  
CREATE INDEX idx_notifications_topic ON notifications(topic);  
CREATE INDEX idx_notifications_event_type ON notifications(event_type);  
CREATE INDEX idx_notifications_created_at ON notifications(created_at DESC);
```

3.11.5 Testing

Query Notifications:

powershell

```
$notifications = Invoke-RestMethod -Uri "http://localhost:8003/api/notifications?user_id=$global:userId  
&limit=10" `   
  -Method GET -Headers @{Authorization="Bearer $global:token"}  
  
$notifications | ConvertTo-Json -Depth 5
```

Expected Output:

json

```
[  
  {  
    "id": "notif-12345",  
    "user_id": "f243ed10-5b07-4e5c-b9ee-bd668ea257ac",  
    "topic": "quiz.generated",
```

```
"event_type": "quiz.generated.v1",
"event_id": "evt-12345",
"payload": {
  "quiz_id": "quiz-12345",
  "document_id": "3af31b69-5a41-49de-b656-1b9b296b7744"
},
"created_at": "2025-01-XXT12:00:00Z"
}
]
```

Get Statistics:

powershell

```
$stats = Invoke-RestMethod -Uri "http://localhost:8003/api/notifications/stats" `
    -Method GET -Headers @{Authorization="Bearer $global:token"}

$stats | ConvertTo-Json
```

Expected Output:

json

```
{
  "total_notifications": 150,
  "by_topic": {
    "document.processed": 50,
    "quiz.generated": 30,
    "audio.generation.completed": 40,
    "audio.transcription.completed": 30
  },
  "by_event_type": {
    "document.processed.v1": 50,
    "quiz.generated.v1": 30,
    "audio.generation.completed.v1": 40,
    "audio.transcription.completed.v1": 30
  }
}
```

4. API Documentation

4.1 Authentication

All protected endpoints require a JWT token in the `Authorization` header:

text

```
Authorization: Bearer <token>
```


Token Format:

- Header: `{"alg": "HS256", "typ": "JWT"}`
- Payload: `{"user_id": "...", "username": "...", "exp": ...}`
- Signature: HMAC-SHA256

4.2 Common Response Formats

Success Response:

json

```
{
  "status": "success",
  "data": { ... }
}
```

Error Response:

json

```
{
  "status": "error",
  "error": {
    "message": "Error description",
    "code": "ERROR_CODE"
  }
}
```

HTTP Status Codes:

- `200 OK` : Success
 - `201 Created` : Resource created
 - `202 Accepted` : Request accepted (async processing)
 - `400 Bad Request` : Invalid request
 - `401 Unauthorized` : Missing or invalid token
 - `403 Forbidden` : Insufficient permissions
 - `404 Not Found` : Resource not found
 - `500 Internal Server Error` : Server error
 - `503 Service Unavailable` : Service temporarily unavailable
-

5. Event-Driven Architecture

5.1 Kafka Topics

Topic	Partitions	Replication	Retention	Purpose
<code>document.uploaded</code>	3	1	7 days	Document upload events
<code>document.processed</code>	3	1	7 days	Document processing complete
<code>notes.generated</code>	3	1	7 days	Notes generation complete
<code>quiz.requested</code>	3	1	7 days	Quiz generation requests
<code>quiz.generated</code>	3	1	7 days	Quiz generation complete
<code>audio.generation.requested</code>	3	1	7 days	TTS generation requests
<code>audio.generation.completed</code>	3	1	7 days	TTS generation complete
<code>audio.transcription.requested</code>	3	1	7 days	STT transcription requests
<code>audio.transcription.completed</code>	3	1	7 days	STT transcription complete
<code>chat.message</code>	3	1	7 days	Chat messages
<code>user.created</code>	3	1	7 days	User registration events

5.2 Event Schema

All events follow the `BaseEvent` schema:

python

```
class BaseEvent(BaseModel):
    event_type: EventType
    event_id: str
    timestamp: datetime
    trace_id: str
```

```
correlation_id: Optional[str]  
schema_version: str = "1.0.0"
```

5.3 Complete Event Flow Diagrams

5.3.1 Document Upload & Processing Flow

1. User uploads document via Aggregator
2. Document Service receives file
3. Upload to S3 (raw file)
4. Save metadata to PostgreSQL (status: UPLOADED)
5. Publish Kafka event: document.uploaded
6. Document Worker consumes event
7. Download from S3
8. Extract text (PDF/DOCX/TXT)
9. Upload extracted text to S3
10. Generate notes using Gemini 1.5 Flash
11. Upload notes to S3
12. Update database (status: PROCESSED)
13. Publish Kafka events: document.processed, notes.generated
14. Notification Service consumes events
15. Log notifications

5.3.2 Quiz Generation Flow

1. User requests quiz via Aggregator
2. Aggregator publishes Kafka event: quiz.requested
3. Quiz Worker consumes event
4. Download document text from S3
5. Call HuggingFace API (Qwen/Qwen2.5-7B-Instruct)
6. Parse JSON response
7. Save quiz to PostgreSQL
8. Upload quiz JSON to S3

9. Publish Kafka event: quiz.generated
10. Notification Service notifies user

5.3.3 Chat with RAG Workflow

1. User sends message via Aggregator
2. Chat Service receives request
3. Generate query embedding (text-embedding-004)
4. Vector search (pgvector) - Find 5 most similar chunks
5. Retrieve context chunks
6. Retrieve conversation history (last 10 messages)
7. Call Gemini 1.5 Flash (context + query + history)
8. Save conversation to database
9. Publish Kafka event: chat.message
10. Notification Service logs event

5.3.4 TTS Generation Flow

1. User requests TTS via Aggregator
2. Aggregator publishes Kafka event: audio.generation.requested
3. TTS Service consumes event
4. Synthesize speech using gTTS
5. Upload audio file to S3
6. Save metadata to MongoDB
7. Publish Kafka event: audio.generation.completed
8. Notification Service notifies user

5.3.5 STT Transcription Flow

1. User requests STT via Aggregator
2. Aggregator publishes Kafka event: audio.transcription.requested
3. STT Service consumes event
4. Download audio file from S3
5. Convert audio to Base64

6. Send to Gemini 1.5 Flash (multimodal API)
 7. Receive transcription text
 8. Save to MongoDB
 9. Publish Kafka event: audio.transcription.completed
 10. Notification Service notifies user
-

6. Database Schemas

6.1 User Service Database

Database: `user_management` (PostgreSQL)

sql

```
CREATE TABLE users (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    username VARCHAR(50) UNIQUE NOT NULL,  
    email VARCHAR(100) UNIQUE NOT NULL,  
    password_hash VARCHAR(255) NOT NULL,  
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),  
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()  
);  
  
CREATE INDEX idx_users_email ON users(email);  
CREATE INDEX idx_users_username ON users(username);
```

6.2 Document Service Database

Database: `document_reader_db` (PostgreSQL)

sql

```
CREATE TABLE documents (  
    id UUID PRIMARY KEY,  
    user_id VARCHAR(64) NOT NULL,  
    filename VARCHAR(255) NOT NULL,  
    original_filename VARCHAR(255),  
    s3_raw_url TEXT,  
    s3_text_url TEXT,  
    s3_notes_url TEXT,  
    status VARCHAR(50) DEFAULT 'UPLOADED',  
    file_size INTEGER,  
    file_type VARCHAR(100),  
    pages INTEGER,  
    text_length INTEGER,  
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),  
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()  
);
```

```
CREATE INDEX idx_documents_user_id ON documents(user_id);
CREATE INDEX idx_documents_status ON documents(status);
```

6.3 Quiz Service Database

Database: `quiz_db` (PostgreSQL)

sql

```
CREATE TABLE quizzes (
  id UUID PRIMARY KEY,
  user_id VARCHAR(64) NOT NULL,
  document_id UUID NOT NULL,
  title VARCHAR(255) NOT NULL,
  difficulty VARCHAR(50),
  question_count INTEGER,
  s3_url TEXT,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE TABLE questions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  quiz_id UUID REFERENCES quizzes(id) ON DELETE CASCADE,
  question_text TEXT NOT NULL,
  options JSONB NOT NULL,
  correct_answer_index INTEGER NOT NULL,
  explanation TEXT,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE INDEX idx_quizzes_user_id ON quizzes(user_id);
CREATE INDEX idx_quizzes_document_id ON quizzes(document_id);
CREATE INDEX idx_questions_quiz_id ON questions(quiz_id);
```

6.4 Chat Service Database

Database: `chat_db` (PostgreSQL with pgvector)

sql

```
-- Enable pgvector extension
CREATE EXTENSION IF NOT EXISTS vector;

-- Conversations table
CREATE TABLE conversations (
  id VARCHAR(36) PRIMARY KEY,
  user_id VARCHAR(64) NOT NULL,
  title VARCHAR(255),
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Messages table
CREATE TABLE messages (
```

```

        id VARCHAR(36) PRIMARY KEY,
        conversation_id VARCHAR(36) REFERENCES conversations(id),
        role VARCHAR(20) NOT NULL,
        content TEXT NOT NULL,
        created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
    );

-- Document embeddings table (for RAG)
CREATE TABLE document_embeddings (
    id VARCHAR(36) PRIMARY KEY,
    document_id VARCHAR(36) NOT NULL,
    content TEXT NOT NULL,
    embedding vector(768),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE INDEX idx_conversations_user_id ON conversations(user_id);
CREATE INDEX idx_messages_conversation_id ON messages(conversation_id);
CREATE INDEX idx_document_embeddings_document_id ON document_embeddings(document_id);
CREATE INDEX idx_document_embeddings_vector ON document_embeddings USING ivfflat (embedding vector_l2_ops);

```

6.5 Notification Service Database

Database: `notification_db` (PostgreSQL)

sql

```

CREATE TABLE notifications (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id VARCHAR(64),
    topic VARCHAR(100) NOT NULL,
    event_type VARCHAR(100) NOT NULL,
    event_id VARCHAR(100) NOT NULL,
    payload JSONB NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE INDEX idx_notifications_user_id ON notifications(user_id);
CREATE INDEX idx_notifications_topic ON notifications(topic);
CREATE INDEX idx_notifications_event_type ON notifications(event_type);
CREATE INDEX idx_notifications_created_at ON notifications(created_at DESC);

```

6.6 TTS Service Database

Database: `tts_service` (MongoDB)

javascript

```

{
  _id: ObjectId,
  userId: String,
  text: String,
  audioUrl: String,

```

```

    status: String, // 'pending', 'completed', 'failed'
    languageCode: String,
    createdAt: Date,
    updatedAt: Date
  }

// Indexes
db.audios.createIndex({ userId: 1 });
db.audios.createIndex({ status: 1 });
db.audios.createIndex({ createdAt: -1 });

```

6.7 STT Service Database

Database: `stt_service` (MongoDB)

javascript

```

{
  _id: ObjectId,
  userId: String,
  audioUrl: String,
  transcription: String,
  confidence: Number,
  status: String, // 'pending', 'completed', 'failed'
  languageCode: String,
  createdAt: Date,
  updatedAt: Date
}

// Indexes
db.transcriptions.createIndex({ userId: 1 });
db.transcriptions.createIndex({ status: 1 });
db.transcriptions.createIndex({ createdAt: -1 });

```

7. SOLID Principles Application

7.1 Single Responsibility Principle (SRP)

Applied Throughout:

- **User Service:** Only handles user authentication and management
- **Document Service:** Only handles document upload and processing
- **Quiz Service:** Only handles quiz generation
- **Chat Service:** Only handles conversational AI
- **TTS/STT Services:** Only handle audio processing
- **Notification Service:** Only handles event logging

Example - Document Service:

python

```
# ✅ Good: Separate classes for different responsibilities
class S3Service:
    """Only handles S3 operations"""
    def upload_file(self, content: bytes, key: str) -> str: ...
    def download_file(self, s3_url: str) -> bytes: ...

class DocumentProcessor:
    """Only handles document processing"""
    def extract_text_from_pdf(self, content: bytes) -> str: ...
    def extract_text_from_docx(self, content: bytes) -> str: ...

class AIService:
    """Only handles AI operations"""
    def generate_notes(self, text: str) -> str: ...
```

7.2 Open/Closed Principle (OCP)

Applied in:

- **Event Models:** Extensible via inheritance
- **Service Interfaces:** Can be extended without modification

Example - Event Models:

python

```
# Base event (closed for modification)
class BaseEvent(BaseModel):
    event_type: EventType
    event_id: str
    timestamp: datetime
    # ...

# Specific events (open for extension)
class DocumentUploadedEvent(BaseEvent):
    document_id: str
    user_id: str
    s3_uri: str
    # ...

class QuizGeneratedEvent(BaseEvent):
    quiz_id: str
    document_id: str
    # ...
```

7.3 Liskov Substitution Principle (LSP)

Applied in:

- **Database Abstractions:** Can swap PostgreSQL implementations
- **S3 Service:** Can swap LocalStack for real S3

Example - S3 Service:

python

```
class S3Service:
    def __init__(self):
        # Works with both LocalStack and real S3
        if self.endpoint_url:
            s3_kwargs['endpoint_url'] = self.endpoint_url
        self.s3_client = boto3.client('s3', **s3_kwargs)
```

7.4 Interface Segregation Principle (ISP)

Applied in:

- **Service Interfaces:** Clients only depend on methods they use
- **Repository Pattern:** Separate interfaces for different operations

Example - Repository Pattern:

python

```
# User Service Repository
class UserRepository:
    def create_user(self, user_data: dict) -> User: ...
    def get_user_by_email(self, email: str) -> User: ...
    def get_user_by_id(self, user_id: str) -> User: ...
    # Only methods needed by User Service
```

7.5 Dependency Inversion Principle (DIP)

Applied in:

- **Dependency Injection:** Services depend on abstractions, not concretions
- **FastAPI Depends:** Dependency injection for database sessions

Example - Dependency Injection:

python

```
# ✅ Good: Depends on abstraction (Session), not concrete implementation
@app.post("/api/documents/upload")
async def upload_document(
    file: UploadFile,
    db: Session = Depends(get_db), # Injected dependency
    user: dict = Depends(get_current_user) # Injected dependency
```

```
):  
    # Use db and user without knowing implementation details  
    ...
```

8. Testing & Verification

8.1 Service Health Checks

All Services:

powershell

```
# User Service  
Invoke-RestMethod -Uri "http://localhost:8000/health"  
  
# Aggregator  
Invoke-RestMethod -Uri "http://localhost:8080/health"  
  
# Document Service  
Invoke-RestMethod -Uri "http://localhost:8002/health"  
  
# Quiz Service  
Invoke-RestMethod -Uri "http://localhost:8004/health"  
  
# Chat Service  
Invoke-RestMethod -Uri "http://localhost:8005/health"  
  
# TTS Service  
Invoke-RestMethod -Uri "http://localhost:8006/health"  
  
# STT Service  
Invoke-RestMethod -Uri "http://localhost:8007/health"  
  
# Notification Service  
Invoke-RestMethod -Uri "http://localhost:8003/health"
```

Expected Output (All Services):

json

```
{  
  "status": "healthy",  
  "service": "<service-name>"  
}
```

8.2 End-to-End Testing Workflow

8.2.1 Complete User Journey: Document Upload → Quiz Generation → Chat

Step 1: Register and Login

powershell

```
# Register
$registerBody = @{
    username = "testuser"
    email = "test@example.com"
    password = "Test123!"
} | ConvertTo-Json

$registerResponse = Invoke-RestMethod -Uri "http://localhost:8000/api/auth/register" `
    -Method POST -Body $registerBody -ContentType "application/json"

# Login
$loginBody = @{
    email = "test@example.com"
    password = "Test123!"
} | ConvertTo-Json

$loginResponse = Invoke-RestMethod -Uri "http://localhost:8000/api/auth/login" `
    -Method POST -Body $loginBody -ContentType "application/json"

$global:token = $loginResponse.access_token
$global:userId = $loginResponse.user_id

Write-Host "✅ Logged in. User ID: $global:userId"
```

Step 2: Upload Document

powershell

```
# Create a test PDF or use existing file
$filePath = "C:\path\to\test-document.pdf"
$fileBytes = [System.IO.File]::ReadAllBytes($filePath)
$boundary = [System.Guid]::NewGuid().ToString()

$bodyLines = @(
    "--$boundary",
    "Content-Disposition: form-data; name=`file`; filename=`test-document.pdf`",
    "Content-Type: application/pdf",
    "",
    [System.Text.Encoding]::GetEncoding("iso-8859-1").GetString($fileBytes),
    "--$boundary--"
) -join "`r`n"

$headers = @{
    Authorization = "Bearer $global:token"
    "Content-Type" = "multipart/form-data; boundary=$boundary"
}

$uploadResponse = Invoke-RestMethod -Uri "http://localhost/api/documents/upload" `
    -Method POST -Body $bodyLines -Headers $headers

$global:documentId = $uploadResponse.document_id
```

```
Write-Host "✅ Document uploaded. Document ID: $global:documentId"
Write-Host "Status: $($uploadResponse.status)"
```

Expected Output:

json

```
{
  "document_id": "3af31b69-5a41-49de-b656-1b9b296b7744",
  "filename": "test-document.pdf",
  "status": "UPLOADED",
  "message": "Document uploaded successfully. Processing will begin shortly."
}
```

Step 3: Wait for Document Processing

powershell

```
# Wait 30 seconds for processing
Write-Host "⌚ Waiting for document processing (30 seconds)..."
Start-Sleep -Seconds 30

# Check document status
$doc = Invoke-RestMethod -Uri "http://localhost/api/documents/$global:documentId" `
    -Method GET -Headers @{Authorization="Bearer $global:token"}

Write-Host "Document Status: $($doc.status)"
Write-Host "Pages: $($doc.pages)"
Write-Host "Text Length: $($doc.text_length)"

if ($doc.status -eq "PROCESSED") {
    Write-Host "✅ Document processed successfully!"
} else {
    Write-Host "⚠ Document still processing. Check logs: docker-compose logs document-worker"
}
```

Step 4: Get Generated Notes

powershell

```
$notes = Invoke-RestMethod -Uri "http://localhost/api/documents/$global:documentId/notes" `
    -Method GET -Headers @{Authorization="Bearer $global:token"}

Write-Host "Notes URL: $($notes.notes_url)"
Write-Host "✅ Notes generated successfully!"
```

Step 5: Request Quiz Generation

powershell

```

$quizRequest = Invoke-RestMethod -Uri "http://localhost/api/quiz/generate?document_id=$global:documentId&difficulty=medium&question_count=10" `
    -Method POST -Headers @{Authorization="Bearer $global:token"}

Write-Host "Request ID: $($quizRequest.request_id)"
Write-Host "Status: $($quizRequest.status)"
Write-Host "🕒 Waiting for quiz generation (60 seconds)..."
Start-Sleep -Seconds 60

```

Step 6: Get Generated Quiz

powershell

```

$quizzes = Invoke-RestMethod -Uri "http://localhost/api/quizzes?user_id=$global:userId&document_id=$global:documentId" `
    -Method GET -Headers @{Authorization="Bearer $global:token"}

if ($quizzes.Count -gt 0) {
    $quiz = $quizzes[0]
    Write-Host "✅ Quiz generated!"
    Write-Host "Quiz ID: $($quiz.id)"
    Write-Host "Title: $($quiz.title)"
    Write-Host "Questions: $($quiz.questions.Count)"

    # Display first question
    if ($quiz.questions.Count -gt 0) {
        $q = $quiz.questions[0]
        Write-Host "`nFirst Question:"
        Write-Host "  $($q.question_text)"
        Write-Host "  Options:"
        for ($i = 0; $i -lt $q.options.Count; $i++) {
            Write-Host "    $i. $($q.options[$i])"
        }
    }
} else {
    Write-Host "⚠ Quiz not yet generated. Check logs: docker-compose logs quiz-worker"
}

```

Step 7: Chat with Document

powershell

```

# First message
$chatBody = @{
    user_id = $global:userId
    document_id = $global:documentId
    message = "What is this document about?"
} | ConvertTo-Json

$chatResponse = Invoke-RestMethod -Uri "http://localhost:8005/api/chat/message" `
    -Method POST -Body $chatBody -ContentType "application/json" `
    -Headers @{Authorization="Bearer $global:token"}

```

```

Write-Host "✅ Chat Response:"
Write-Host $chatResponse.response
Write-Host "`nConversation ID: $($chatResponse.conversation_id)"
Write-Host "Sources Count: $($chatResponse.sources_count)"

$global:conversationId = $chatResponse.conversation_id

# Follow-up message
$followUpBody = @{
    user_id = $global:userId
    document_id = $global:documentId
    message = "Can you explain the main concepts in more detail?"
    conversation_id = $global:conversationId
} | ConvertTo-Json

$followUpResponse = Invoke-RestMethod -Uri "http://localhost:8005/api/chat/message" `
    -Method POST -Body $followUpBody -ContentType "application/json" `
    -Headers @{Authorization="Bearer $global:token"}

Write-Host "`n✅ Follow-up Response:"
Write-Host $followUpResponse.response

```

Expected Output:

json

```

{
  "response": "This document discusses machine learning fundamentals, including supervised and unsupervised learning algorithms, neural networks, and their applications...",
  "conversation_id": "conv-12345",
  "sources_count": 5
}

```

8.3 TTS/STT Testing Workflow

8.3.1 Text-to-Speech Generation

powershell

```

# Request TTS
$ttsBody = @{
    text = "Hello, this is a test of the text to speech system. The quick brown fox jumps over the lazy dog."
    languageCode = "en"
} | ConvertTo-Json

$ttsResponse = Invoke-RestMethod -Uri "http://localhost/api/tts/synthesize" `
    -Method POST -Body $ttsBody -ContentType "application/json" `
    -Headers @{Authorization="Bearer $global:token"}

Write-Host "Request ID: $($ttsResponse.request_id)"
Write-Host "🕒 Waiting for TTS generation (10 seconds)..."
Start-Sleep -Seconds 10

```

```
# Check MongoDB for results
docker-compose exec -T mongodb mongosh --quiet --eval "use tts_service; db.audios.find().sort({createdAt: -1}).limit(1).pretty()"
```

Expected MongoDB Output:

json

```
{
  "_id": ObjectId("..."),
  "userId": "f243ed10-5b07-4e5c-b9ee-bd668ea257ac",
  "text": "Hello, this is a test...",
  "audioUrl": "http://localstack:4566/document-reader-storage-dev/audio/...",
  "status": "completed",
  "languageCode": "en",
  "createdAt": ISODate("2025-01-XXT12:00:00Z")
}
```

8.3.2 Speech-to-Text Transcription

powershell

```
# Use the audio URL from TTS generation
$audioS3Uri = "http://localstack:4566/document-reader-storage-dev/audio/1764965514866-tts/f243ed10-5b07-4e5c-b9ee-bd668ea257ac/1764965514866.mp3"

# Request STT
$sttResponse = Invoke-RestMethod -Uri "http://localhost/api/stt/transcribe?audio_id=test-audio&s3_uri=$([System.Web.HttpUtility]::UrlEncode($audioS3Uri))" `
  -Method POST -Headers @{Authorization="Bearer $global:token"}

Write-Host "Request ID: $($sttResponse.request_id)"
Write-Host "🕒 Waiting for transcription (15 seconds)..."
Start-Sleep -Seconds 15

# Check MongoDB for results
docker-compose exec -T mongodb mongosh --quiet --eval "use stt_service; db.transcriptions.find().sort({createdAt: -1}).limit(1).pretty()"
```

Expected MongoDB Output:

json

```
{
  "_id": ObjectId("..."),
  "userId": "f243ed10-5b07-4e5c-b9ee-bd668ea257ac",
  "audioUrl": "http://localstack:4566/document-reader-storage-dev/audio/...",
  "transcription": "Hello, this is a test of the text to speech system. The quick brown fox jumps over the lazy dog.",
  "confidence": 0.95,
}
```



```
"status": "completed",
"languageCode": "en-US",
"createdAt": ISODate("2025-01-XXT12:00:15Z")
}
```

8.4 Monitoring and Logs

8.4.1 View Service Logs

powershell

```
# All services
docker-compose logs -f

# Specific service
docker-compose logs -f document-worker
docker-compose logs -f quiz-worker
docker-compose logs -f chat-worker
docker-compose logs -f tts-service
docker-compose logs -f stt-service
docker-compose logs -f notification-service

# Last 50 lines
docker-compose logs --tail 50 document-worker

# Filter by pattern
docker-compose logs document-worker | Select-String -Pattern "ERROR|WARNING|Successfully"
```

8.4.2 Check Kafka Topics

powershell

```
# List all topics
docker-compose exec kafka kafka-topics --list --bootstrap-server localhost:9092

# Describe topic
docker-compose exec kafka kafka-topics --describe --topic document.processed --bootstrap-server localhost:9092

# Consume messages from topic
docker-compose exec kafka kafka-console-consumer --bootstrap-server localhost:9092 --topic document.processed --from-beginning
```

8.4.3 Check Database Status

powershell

```
# PostgreSQL databases
docker-compose exec user-db psql -U platformadmin -d user_management -c "SELECT COUNT(*) FROM users;"
docker-compose exec document-db psql -U postgres -d document_reader_db -c "SELECT COUNT(*) FROM documents;"
docker-compose exec quiz-db psql -U postgres -d quiz_db -c "SELECT COUNT(*) FROM quizzes;"
docker-compose exec chat-db psql -U postgres -d chat_db -c "SELECT COUNT(*) FROM document_embeddings;"

# MongoDB
docker-compose exec mongodb mongosh --quiet --eval "use tts_service; db.audios.countDocuments()"
docker-compose exec mongodb mongosh --quiet --eval "use stt_service; db.transcriptions.countDocuments()"
```

9. Containerization & Deployment

9.1 Docker Architecture

9.1.1 Multi-Stage Builds

All Python services use **multi-stage Docker builds** to reduce image size:

Stage 1: Builder

- Installs build dependencies (gcc, python3-dev)
- Installs Python packages to `/install` location
- Uses BuildKit cache mounts for faster rebuilds

Stage 2: Runtime

- Copies only installed packages from builder
- Installs runtime-only system dependencies
- Copies application code
- Creates non-root user for security

Example - Quiz Service Dockerfile:

dockerfile

```
# STAGE 1: Builder
FROM python:3.11-slim as builder

WORKDIR /app
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1

# Install build dependencies
RUN apt-get update && \
    apt-get install -y --no-install-recommends gcc python3-dev && \
    rm -rf /var/lib/apt/lists/*
```

```

# Install python dependencies to /install location
COPY ./platform/quiz-service/requirements.txt .
RUN --mount=type=cache,target=/root/.cache/pip \
    pip install --no-cache-dir --prefix=/install -r requirements.txt

# Install shared library
COPY ./shared /shared
RUN --mount=type=cache,target=/root/.cache/pip \
    pip install --no-cache-dir --prefix=/install /shared

# STAGE 2: Runtime (Final Image)
FROM python:3.11-slim

WORKDIR /app
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 PYTHONPATH=/app/src

# Copy installed python packages from builder
COPY --from=builder /install /usr/local

# Install runtime-only system deps
RUN apt-get update && \
    apt-get install -y --no-install-recommends curl && \
    rm -rf /var/lib/apt/lists/*

# Copy app code
COPY ./platform/quiz-service/src ./src
COPY ./shared /shared

# Create non-root user
RUN useradd -m -u 1000 appuser && chown -R appuser:appuser /app
USER appuser

# Health check
HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3 \
    CMD curl -f http://localhost:8004/health || exit 1

EXPOSE 8004
CMD ["uvicorn", "src.main:app", "--host", "0.0.0.0", "--port", "8004"]

```

9.1.2 Node.js Services (TTS/STT)

TTS Service Dockerfile:

dockerfile

```

FROM node:18-alpine

WORKDIR /app

# Copy package files
COPY ./tts-service/package*.json ./

# Install dependencies (production only)
RUN npm install --only=production

```

```
# Copy application code
COPY ./tts-service/src ./src

# Create non-root user
RUN addgroup -g 1000 appuser && \
  adduser -D -u 1000 -G appuser appuser && \
  chown -R appuser:appuser /app
USER appuser

# Health check
HEALTHCHECK --interval=30s --timeout=3s --start-period=40s \
  CMD node -e "require('http').get('http://localhost:8006/health', (r) => {process.exit(r.statusCode === 200 ? 0 : 1)})"

EXPOSE 8006
CMD ["npm", "start"]
```

9.1.3 Graceful Shutdown

Python Services (FastAPI):

python

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    # Startup
    yield
    # Shutdown: FastAPI handles cleanup automatically
```

Node.js Services:

javascript

```
// Graceful shutdown handler
const gracefulShutdown = async (signal) => {
  logger.info(`${signal} received. Starting graceful shutdown...`);

  // 1. Stop accepting new HTTP requests
  if (server) {
    server.close(() => {
      logger.info("HTTP server closed.");
    });
  }

  // 2. Disconnect Kafka
  await disconnectKafka();

  // 3. Disconnect MongoDB
  if (mongoose.connection.readyState === 1) {
    await db.disconnect();
    logger.info("MongoDB disconnected.");
  }

  logger.info("Graceful shutdown complete.");
}
```

```

    process.exit(0);
};

process.on("SIGTERM", () => gracefulShutdown("SIGTERM"));
process.on("SIGINT", () => gracefulShutdown("SIGINT"));

```

9.2 Docker Compose Configuration

9.2.1 Service Dependencies

yaml

```

services:
  # Infrastructure
  zookeeper:
    image: confluentinc/cp-zookeeper:7.4.0
    # ...

  kafka:
    depends_on:
      - zookeeper
    # ...

  # Databases
  user-db:
    image: postgres:15
    # ...

  document-db:
    image: postgres:15
    # ...

  quiz-db:
    image: postgres:15
    # ...

  chat-db:
    image: pgvector/pgvector:pg16 # CRITICAL: pgvector extension
    # ...

  mongodb:
    image: mongo:7.0
    # ...

  # Services with dependencies
  user-service:
    depends_on:
      user-db:
        condition: service_healthy
      kafka:
        condition: service_started
    # ...

  document-worker:
    depends_on:
      document-service:

```

```

        condition: service_started
document-db:
    condition: service_healthy
kafka:
    condition: service_started
localstack:
    condition: service_healthy
# ...

```

9.2.2 Environment Variables

Common Variables:

- `DATABASE_URL` : PostgreSQL connection string
- `KAFKA_BOOTSTRAP_SERVERS` : Kafka broker address
- `S3_BUCKET_NAME` : S3 bucket name
- `AWS_ACCESS_KEY_ID` : AWS credentials (test for LocalStack)
- `AWS_SECRET_ACCESS_KEY` : AWS credentials (test for LocalStack)
- `S3_ENDPOINT_URL` : LocalStack endpoint (<http://localstack:4566>)
- `JWT_SECRET` : JWT signing secret
- `SERVICE_NAME` : Service identifier for logging
- `LOG_LEVEL` : Logging level (INFO, DEBUG, etc.)

Service-Specific Variables:

- `GOOGLE_API_KEY` : Google Gemini API key (Document Service)
- `CHAT_GOOGLE_API_KEY` : Google Gemini API key (Chat Service)
- `HUGGINGFACE_API_KEY` : HuggingFace API token (Quiz Service)
- `HUGGINGFACE_MODEL` : Model ID (default: Qwen/Qwen2.5-7B-Instruct)
- `MONGODB_URI` : MongoDB connection string (TTS/STT Services)

9.3 Image Versioning

9.3.1 Version Tagging Script

PowerShell Script (`scripts/tag_and_push.ps1`):

powershell

```

param(
    [string]$Version = "latest",

```

```

    [string]$RegistryUrl = ""
)

$services = @(
    "user-service",
    "aggregator",
    "document-service",
    "document-worker",
    "quiz-service",
    "quiz-worker",
    "chat-service",
    "chat-worker",
    "notification-service",
    "tts-service",
    "stt-service"
)

foreach ($service in $services) {
    $localImage = "cloud-{$service}"
    $remoteImage = "${RegistryUrl}/{$service}:${Version}"
    $latestImage = "${RegistryUrl}/{$service}:latest"

    docker tag $localImage $remoteImage
    docker tag $localImage $latestImage
    docker push $remoteImage
    docker push $latestImage
}

```

Usage:

powershell

```

.\scripts\tag_and_push.ps1 -Version v1.0.0 -RegistryUrl 123456789012.dkr.ecr.us-east-1.amazonaws.com

```

9.4 Health Checks

All services implement health check endpoints:

Python Services:

python

```

@app.get("/health")
def health():
    return {"status": "healthy", "service": "service-name"}

```

Node.js Services:

javascript

```
app.get("/health", (req, res) => {
  res.status(200).json({ status: "healthy", service: "service-name" });
});
```

Docker Health Checks:

yaml

```
healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost:8000/health"]
  interval: 30s
  timeout: 10s
  retries: 3
```

10. Complete Workflows

10.1 Document Processing Workflow

Complete Flow:

1. User uploads document via Aggregator
2. Document Service receives file
3. Upload to S3 (raw file)
4. Save metadata to PostgreSQL (status: UPLOADED)
5. Publish Kafka event: document.uploaded
6. Document Worker consumes event
7. Download from S3
8. Extract text (PDF/DOCX/TXT)
9. Upload extracted text to S3
10. Generate notes using Gemini 1.5 Flash
11. Upload notes to S3
12. Update database (status: PROCESSED)
13. Publish Kafka events: document.processed, notes.generated
14. Downstream services consume events:
 - Quiz Worker: Stores reference for quiz generation
 - Chat Worker: Vectorizes document for RAG

- Notification Service: Logs notification

Timeline:

- **T+0s:** User uploads document
- **T+1s:** Document uploaded to S3, event published
- **T+2s:** Document Worker starts processing
- **T+10s:** Text extracted, uploaded to S3
- **T+20s:** Notes generated, uploaded to S3
- **T+25s:** Events published, services notified
- **T+30s:** Document ready for quiz/chat

10.2 Quiz Generation Workflow

Complete Flow:

1. User requests quiz via Aggregator
2. Aggregator publishes Kafka event: quiz.requested
3. Quiz Worker consumes event
4. Download document text from S3
5. Call HuggingFace API (Qwen/Qwen2.5-7B-Instruct)
6. Parse JSON response
7. Save quiz to PostgreSQL
8. Upload quiz JSON to S3
9. Publish Kafka event: quiz.generated
10. Notification Service consumes event
11. Alternative trigger: Auto-generate quiz on document.processed event

Timeline:

- **T+0s:** User requests quiz
- **T+1s:** Event published to Kafka
- **T+2s:** Quiz Worker starts processing
- **T+5s:** Document text downloaded
- **T+10s:** HuggingFace API called
- **T+45s:** Quiz generated, saved to DB

- **T+50s:** Event published, user notified
- **T+60s:** Quiz available via API

10.3 Chat with RAG Workflow

Complete Flow:

1. User sends message via Aggregator
2. Chat Service receives request
3. Generate query embedding (text-embedding-004)
4. Vector search (pgvector) - Find 5 most similar chunks
5. Retrieve context chunks
6. Retrieve conversation history (last 10 messages)
7. Call Gemini 1.5 Flash (context + query + history)
8. Save conversation to database
9. Publish Kafka event: chat.message
10. Notification Service logs event

Background Processing:

- Chat Worker consumes document.processed events
- Downloads document text from S3
- Splits text into chunks (1000 chars, 200 overlap)
- Generates embeddings (text-embedding-004)
- Stores vectors in pgvector

Timeline:

- **T+0s:** User sends message
- **T+0.5s:** Query embedding generated
- **T+1s:** Vector search completed
- **T+1.5s:** Context chunks retrieved
- **T+2s:** Gemini API called
- **T+4s:** Response received
- **T+4.5s:** Conversation saved
- **T+5s:** Response returned to user

10.4 TTS/STT Workflows

TTS Generation Flow:

1. User requests TTS via Aggregator
2. Aggregator publishes Kafka event: `audio.generation.requested`
3. TTS Service consumes event
4. Synthesize speech using gTTS
5. Upload MP3 to S3
6. Save metadata to MongoDB
7. Publish Kafka event: `audio.generation.completed`
8. Notification Service notifies user

STT Transcription Flow:

1. User requests STT via Aggregator
2. Aggregator publishes Kafka event: `audio.transcription.requested`
3. STT Service consumes event
4. Download audio file from S3
5. Convert audio to Base64
6. Call Gemini 1.5 Flash (multimodal API)
7. Receive transcription text
8. Save to MongoDB
9. Publish Kafka event: `audio.transcription.completed`
10. Notification Service notifies user

Integration:

- TTS audio files can be used as STT input
 - Supports validation of TTS quality through transcription
-

11. Troubleshooting & Known Issues

11.1 Common Issues

11.1.1 Kafka Connection Errors

Symptom:

text

```
kafka.errors.KafkaError: Unable to bootstrap from [('kafka', 9092)]
```

Solution:

powershell

```
# Check Kafka is running
docker-compose ps kafka

# Check Kafka logs
docker-compose logs kafka

# Restart Kafka
docker-compose restart kafka

# Wait for Kafka to be ready
docker-compose exec kafka kafka-broker-api-versions --bootstrap-server localhost:9092
```

11.1.2 Database Connection Errors

Symptom:

text

```
sqlalchemy.exc.OperationalError: could not connect to server
```

Solution:

powershell

```
# Check database is running
docker-compose ps user-db

# Check database logs
docker-compose logs user-db

# Test connection
docker-compose exec user-db psql -U platformadmin -d user_management -c "SELECT 1;"

# Restart database
docker-compose restart user-db
```

11.1.3 S3/LocalStack Errors

Symptom:

text

```
botocore.exceptions.ClientError: The specified bucket does not exist
```

Solution:

powershell

```
# Check LocalStack is running
docker-compose ps localstack

# Create bucket
docker-compose exec localstack aws --endpoint-url=http://localhost:4566 s3 mb s3://document-reader-storage-dev

# List buckets
docker-compose exec localstack aws --endpoint-url=http://localhost:4566 s3 ls
```

11.1.4 HuggingFace API Errors

Symptom:

text

```
400 Bad Request: The requested model '...' is not supported
```

Solution:

- Check `HUGGINGFACE_MODEL` environment variable
- Verify model is available: <https://huggingface.co/models>
- Use `Qwen/Qwen2.5-7B-Instruct` (confirmed working)
- Check API token is valid

11.1.5 Gemini API Errors

Symptom:

text

```
404 models/gemini-1.5-flash is not found
```

Solution:

- Check `GOOGLE_API_KEY` or `CHAT_GOOGLE_API_KEY` is set

- Verify API key is valid
- Check model name is correct: `models/gemini-1.5-flash`
- Use model discovery (already implemented in Chat Service)

11.2 Performance Optimization

11.2.1 Database Indexing

Ensure all indexes are created:

sql

```
-- User Service
CREATE INDEX IF NOT EXISTS idx_users_email ON users(email);
CREATE INDEX IF NOT EXISTS idx_users_username ON users(username);

-- Document Service
CREATE INDEX IF NOT EXISTS idx_documents_user_id ON documents(user_id);
CREATE INDEX IF NOT EXISTS idx_documents_status ON documents(status);

-- Quiz Service
CREATE INDEX IF NOT EXISTS idx_quizzes_user_id ON quizzes(user_id);
CREATE INDEX IF NOT EXISTS idx_quizzes_document_id ON quizzes(document_id);

-- Chat Service (pgvector)
CREATE INDEX IF NOT EXISTS idx_document_embeddings_vector
ON document_embeddings USING ivfflat (embedding vector_l2_ops);
```

11.2.2 Kafka Consumer Groups

Monitor consumer lag:

powershell

```
docker-compose exec kafka kafka-consumer-groups --bootstrap-server localhost:9092 --list
docker-compose exec kafka kafka-consumer-groups --bootstrap-server localhost:9092 --group quiz-service-group --describe
```

11.2.3 S3 Performance

Use presigned URLs for large files:

python

```
# Generate presigned URL (valid for 1 hour)
presigned_url = s3_service.generate_presigned_url(s3_key, expiration=3600)
```

11.3 Debugging Commands

11.3.1 Check Service Status

powershell

```
# All services
docker-compose ps

# Specific service logs
docker-compose logs --tail 100 -f <service-name>

# Service health
Invoke-RestMethod -Uri "http://localhost:<port>/health"
```

11.3.2 Check Kafka Messages

powershell

```
# Consume from topic
docker-compose exec kafka kafka-console-consumer \
  --bootstrap-server localhost:9092 \
  --topic document.processed \
  --from-beginning

# Produce test message
docker-compose exec kafka kafka-console-producer \
  --bootstrap-server localhost:9092 \
  --topic test-topic
```

11.3.3 Check Database Contents

powershell

```
# PostgreSQL
docker-compose exec <db-service> psql -U <user> -d <database> -c "SELECT COUNT(*) FROM <table>;"

# MongoDB
docker-compose exec mongodb mongosh --quiet --eval "use <database>; db.<collection>.countDocuments()"
```

12. Architecture Compliance Checklist

12.1 SOLID Principles

Principle	Status	Implementation
Single Responsibility	✓	Each service has one clear purpose
Open/Closed	✓	Event models extensible via inheritance
Liskov Substitution	✓	S3 service works with LocalStack and real S3
Interface Segregation	✓	Repository pattern with focused interfaces
Dependency Inversion	✓	Dependency injection via FastAPI Depends

12.2 Microservices Principles ✓

Principle	Status	Implementation
Service Independence	✓	Each service can be deployed independently
Storage Isolation	✓	Each service has its own database
API Gateway Pattern	✓	Aggregator as BFF
Event-Driven	✓	Kafka for async communication
Stateless Services	✓	No session state, JWT for auth

12.3 Containerization Requirements ✓

Requirement	Status	Implementation
Multi-Stage Builds	✓	All Python services
Health Checks	✓	All services have <code>/health</code> endpoints
Graceful Shutdown	✓	Python (FastAPI lifespan) + Node.js (SIGTERM handlers)
Non-Root Users	✓	All containers run as non-root
Version Tagging	✓	Scripts for semantic versioning
Official Base Images	✓	<code>python:3.11-slim</code> , <code>node:18-alpine</code>

12.4 Security ✓

Security Feature	Status	Implementation
JWT Authentication	✓	All protected endpoints
Password Hashing	✓	bcrypt with salt
Environment Variables	✓	No hardcoded secrets
HTTPS/TLS	✓	Ready for production (ALB + ACM)
CORS Configuration	✓	Configurable origins

☐	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
☐	 cloud	-	-	-	16.23%	1 hour ago	   
☐	 tts-service	a827e7b5d9f0	cloud-tts-service	8006:8006 	2.68%	1 hour ago	   
☐	 mongodb	c6bafcab6c15	mongo:7.0	27017:27017 	0.4%	1 hour ago	   
☐	 aggregator-1	45644303adc4	cloud-aggregator	8080:8080 	0.31%	1 hour ago	   
☐	 chat-service	22907fad63eb	cloud-chat-service	8005:8005 	0.26%	1 hour ago	   
☐	 chat-worker	b5e4ebb61c7b	cloud-chat-worker		0.29%	1 hour ago	   
☐	 chat-db	9560ca016ca4	pgvector/pgvector:pg16	5436:5432 	1.7%	1 hour ago	   
☐	 quiz-worker	905d5332c552	cloud-quiz-worker		0.24%	1 hour ago	   
☐	 quiz-service-1	b22fa59d6dde	cloud-quiz-service	8004:8004 	0.11%	1 hour ago	   
☐	 document-worker	90cf24741a58	cloud-document-worker		0.31%	1 hour ago	   
☐	 document-service	4e35e4043982	cloud-document-service	8002:8000 	0.12%	1 hour ago	   
☐	 localstack	36c54f69af6e	localstack/localstack:3.0.2	4566:4566 	0.02%	1 hour ago	   
☐	 api-gateway-1	027f0f5348e4	nginx:alpine	80:80 	0%	1 hour ago	   
☐	 notification-service-1	33d86cc16d64	cloud-notification-service	8003:8003 	0.26%	1 hour ago	   
☐	 user-service-1	2212f57a07fd	cloud-user-service	8000:8000 	0.17%	1 hour ago	   
☐	 notification-db-1	5d08cb557671	postgres:15	5434:5432 	0.54%	1 hour ago	   

Showing 22 items

Container CPU usage 		Container memory usage 		Show charts			
33.91% / 1600% (16 CPUs available)		2.29GB / 15.08GB					
Search		 ☐ Only show running containers					
☐	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
☐	 chat-db	9560ca016ca4	pgvector/pgvector:pg16	5436:5432 	5.89%	1 hour ago	   
☐	 quiz-worker	905d5332c552	cloud-quiz-worker		0.2%	1 hour ago	   
☐	 quiz-service-1	b22fa59d6dde	cloud-quiz-service	8004:8004 	0.1%	1 hour ago	   
☐	 document-worker	90cf24741a58	cloud-document-worker		0.22%	1 hour ago	   
☐	 document-service	4e35e4043982	cloud-document-service	8002:8000 	0.12%	1 hour ago	   
☐	 localstack	36c54f69af6e	localstack/localstack:3.0.2	4566:4566 	0.03%	1 hour ago	   
☐	 api-gateway-1	027f0f5348e4	nginx:alpine	80:80 	0%	1 hour ago	   
☐	 notification-service-1	33d86cc16d64	cloud-notification-service	8003:8003 	0.8%	1 hour ago	   
☐	 user-service-1	2212f57a07fd	cloud-user-service	8000:8000 	0.11%	1 hour ago	   
☐	 notification-db-1	5d08cb557671	postgres:15	5434:5432 	4.37%	1 hour ago	   
☐	 user-db-1	c229d09cfea	postgres:15	5432:5432 	4.56%	1 hour ago	   
☐	 kafka-1	655550e98e11	confluentinc/cp-kafka:7.4.0	29092:29092  Show all ports (2)	1.5%	1 hour ago	   
☐	 document-db-1	60e274315b6e	postgres:15	5433:5432 	5.25%	1 hour ago	   
☐	 quiz-db-1	1c30992804a4	postgres:15	5435:5432 	5.04%	1 hour ago	   
☐	 zookeeper-1	daf18c89aa78	confluentinc/cp-zookeeper:7.4.0		0.13%	1 hour ago	   
☐	 stt-service	8326b93692ab	cloud-stt-service	8007:8007 	2.3%	1 hour ago	   

Showing 22 items